Senior AI Engineering Curriculum

Week 2 · Day 10: Attention Optimizations

Flash Attention, Paged KV Internals & Advanced KV Cache Tuning

---

Goal: Understand the three memory tricks that make production inference 5-10x faster than naive HuggingFace: IO-aware tiling in Flash Attention, the physical block management internals of PagedAttention, and the suite of advanced KV cache tuning options in vLLM -- including prefix caching, FP8 KV cache, and memory utilization tuning. Profile each layer and measure the combined speedup.

Prerequisite: Day 9 (vLLM + continuous batching + PagedAttention concepts). Today goes one level deeper into the algorithmic and hardware details behind each optimization. These are the mechanisms you need to understand to debug OOM errors, explain latency regressions, and tune production serving stacks.

PART A — Flash Attention: IO-Aware Tiling

The Memory Bandwidth Bottleneck in Standard Attention

Standard attention (as described in 'Attention Is All You Need') computes the full N x N attention score matrix explicitly in GPU HBM (High Bandwidth Memory). For a sequence of N tokens with embedding dimension d, this requires:

```
# Standard attention memory complexity:
#
#   Q, K, V tensors:    3 x N x d   elements  (read from HBM once)
#   QK^T score matrix:  N x N       elements  (written to HBM, then read back)
#   softmax(QK^T):      N x N       elements  (written to HBM, then read back)
#   Output:             N x d       elements  (written to HBM)
#
# Total HBM reads/writes = O(N^2 + Nd)
#
# For N=8192, d=128 (LLaMA-3 8B per-head dimensions):
#   QK^T matrix alone = 8192 x 8192 x 2 bytes (FP16) = 134 MB per head per layer
#   LLaMA-3 8B has 32 layers x 32 heads = 1024 attention heads
#   Total QK^T per forward pass = 134 MB x 1024 = ~137 GB of HBM traffic!
#
# A100 HBM bandwidth = 2 TB/s
# Time just for QK^T reads/writes = 137 GB / 2000 GB/s = 68ms per forward pass
# This is the bottleneck -- compute (FLOPS) is NOT the issue.

# Key insight: attention is MEMORY BANDWIDTH BOUND, not compute bound.
# The solution is to avoid materialising the N x N matrix in HBM entirely.
```

The GPU Memory Hierarchy

Memory Level	Size (A100)	Bandwidth	Latency	Role in Attention
Registers	256 KB/SM	~20 TB/s	~1 cycle	Arithmetic operands

SRAM (L1/shared)	192 KB/SM	~19 TB/s	~5 cycles	FA tile accumulator
L2 Cache	40 MB	~5 TB/s	~100 cycles	Intermediate results
HBM (VRAM)	40-80 GB	~2 TB/s	~300 cycles	Q, K, V, output storage
CPU DRAM	512 GB+	~50 GB/s (NVL)	~1000 cycles	KV swap (vLLM preempt)

Standard attention repeatedly moves data between HBM (slow, 2 TB/s) and SRAM (fast, 19 TB/s). Flash Attention keeps intermediate results in SRAM and only reads/writes HBM once per tile.

Flash Attention: IO-Aware Tiling Algorithm

Flash Attention (Dao et al., 2022) reorders the attention computation to be **IO-aware**: it processes Q, K, V in small tiles that fit entirely in SRAM, computes the full softmax incrementally using an online normalisation trick, and only writes the final output back to HBM. The N x N matrix is never materialised.

```
# Flash Attention tiling (conceptual pseudocode):
#
# SRAM tile sizes: Br (rows of Q) x Bc (cols of K/V)
# Chosen so that one Q tile + one K tile + one V tile fits in SRAM
#
# Algorithm:
#   Load Q tile[i] into SRAM                        (1 HBM read per tile)
#   For each K tile[j], V tile[j]:
#       Load K[j], V[j] into SRAM                   (1 HBM read per tile)
#       Compute S[i,j] = Q[i] @ K[j]^T              (in SRAM -- fast)
#       Update running max m[i] = max(m[i], rowmax(S[i,j]))
#       Update running sum  l[i] (online softmax denominator)
#       Accumulate output O[i] += softmax(S[i,j]) @ V[j]  (in SRAM)
#   Write O[i] to HBM                               (1 HBM write per tile)
#
# HBM reads:  O(N * d)  -- Q, K, V read once in tiles
# HBM writes: O(N * d)  -- output written once
# vs standard: O(N^2)   -- N x N score matrix read/written multiple times
#
# For N=8192, d=128:
#   Standard: ~137 GB HBM traffic (just for QK^T)
#   Flash:    ~8 GB HBM traffic (Q + K + V + O each read/written once)
#   Speedup:  ~17x reduction in HBM IO -> ~2-4x wall-clock speedup

# Online softmax trick (enables tiling without full N x N):
#   Standard: softmax(x)_i = exp(x_i) / sum(exp(x_j))   -- needs full row
#   Online:   Track running max m and sum l, correct as new tiles arrive
#             Final: O = diag(l)^-1 * O_accumulated
```

Flash Attention Versions

Version	Year	Key Innovation	Speedup vs Std
FlashAttention	2022	IO-aware tiling, online softmax, no N^2 materialise	2-4x prefill
FlashAttention-2	2023	Better work partitioning across warps, less sync	2x over FA1
FlashAttention-3	2024	Async WGMMA + TMA for H100, FP8 support	2x over FA2 on H100

FlashInfer	2024	Ragged batching, page-table aware, vLLM backend	Best decode throughput

Practical Implementation -- Flash Attention

A.1 Install and Enable Flash Attention

```
# Flash Attention requires CUDA 11.6+ and an Ampere+ GPU (A100, RTX 30xx+)
pip install flash-attn --no-build-isolation

# Verify installation
python -c "import flash_attn; print(flash_attn.__version__)"

# FlashInfer (used by vLLM 0.4+ as the default decode backend)
pip install flashinfer -i https://flashinfer.ai/whl/cu121/torch2.3/
```

A.2 Enable in HuggingFace Transformers

```
from transformers import AutoModelForCausalLM, AutoTokenizer
import torch, time

MODEL_ID = "meta-llama/Meta-Llama-3-8B-Instruct"

def load_and_benchmark(attn_impl: str, label: str, n_tokens: int = 200):
    torch.cuda.reset_peak_memory_stats()
    model = AutoModelForCausalLM.from_pretrained(
        MODEL_ID,
        torch_dtype=torch.float16,
        device_map="cuda",
        attn_implementation=attn_impl,   # "eager" | "sdpa" | "flash_attention_2"
    )
    model.eval()
    tok    = AutoTokenizer.from_pretrained(MODEL_ID)
    prompt = "Explain the transformer architecture in detail. " * 8   # ~120 tokens
    inputs = tok(prompt, return_tensors="pt").to("cuda")

    # Warm-up
    with torch.no_grad():
        model.generate(**inputs, max_new_tokens=20,
                       pad_token_id=tok.eos_token_id)
    torch.cuda.synchronize()

    # Timed run
    t0 = time.perf_counter()
    with torch.no_grad():
        out = model.generate(**inputs, max_new_tokens=n_tokens,
                             do_sample=False,
                             pad_token_id=tok.eos_token_id)
    torch.cuda.synchronize()
    elapsed  = time.perf_counter() - t0
    new_toks = out.shape[1] - inputs["input_ids"].shape[1]
    vram_gb  = torch.cuda.max_memory_allocated() / 1e9

    print(f"{label:<30} {new_toks:>8} tok  {elapsed:>8.2f}s  "
```

```
          f"{new_toks/elapsed:>8.1f} tok/s  {vram_gb:>7.2f} GB")
    del model; torch.cuda.empty_cache()

print(f"{'Implementation':<30} {'Tokens':>8}      {'Time':>8}  "
      f"{'Tok/s':>10}  {'VRAM':>9}")
print("-" * 72)
load_and_benchmark("eager",            "Standard Attention (eager)")
load_and_benchmark("sdpa",             "PyTorch SDPA (fused)")
load_and_benchmark("flash_attention_2","Flash Attention 2")
```

A.3 Profile Memory Bandwidth Before vs After Flash Attention

```
# Use PyTorch profiler to measure HBM bandwidth consumed by attention
import torch
from torch.profiler import profile, record_function, ProfilerActivity
from transformers import AutoModelForCausalLM, AutoTokenizer

MODEL_ID = "meta-llama/Meta-Llama-3-8B"
tok      = AutoTokenizer.from_pretrained(MODEL_ID)
prompt   = "The history of attention mechanisms in neural networks"
inputs   = tok(prompt, return_tensors="pt").to("cuda")

for attn_impl in ["eager", "flash_attention_2"]:
    model = AutoModelForCausalLM.from_pretrained(
        MODEL_ID, torch_dtype=torch.float16,
        device_map="cuda", attn_implementation=attn_impl
    )
    model.eval()

    with profile(
        activities=[ProfilerActivity.CUDA],
        record_shapes=True,
        with_flops=True,
        with_stack=False,
        profile_memory=True,
    ) as prof:
        with record_function("attention_forward"):
            with torch.no_grad():
                model(**inputs)

    # Print top ops by CUDA time
    print(f"
=== {attn_impl} ===")
    print(prof.key_averages().table(
        sort_by="cuda_time_total", row_limit=10
    ))

    # Summary
    events = prof.key_averages()
    total_cuda_ms = sum(e.cuda_time_total for e in events) / 1000
    total_mem_mb  = sum(e.self_cuda_memory_usage for e in events
                        if e.self_cuda_memory_usage > 0) / 1e6
    print(f"Total CUDA time : {total_cuda_ms:.2f} ms")
    print(f"Peak memory IO  : {total_mem_mb:.1f} MB")

    del model; torch.cuda.empty_cache()
```

A.4 Flash Attention in vLLM (Auto-Enabled)

```
# vLLM automatically selects the best attention backend:
# - FlashInfer    : default for decode on Ampere+ GPUs (vLLM >= 0.4)
# - Flash Attention 2 : prefill on Ampere+ GPUs
# - xFormers      : fallback for older GPUs
# - PyTorch SDPA  : CPU fallback

# You can inspect which backend vLLM selected at startup:
vllm serve meta-llama/Meta-Llama-3-8B-Instruct --dtype float16 2>&1 | grep -i "attention"
# Look for: "Using FlashInfer backend" or "Using Flash Attention backend"

# Force a specific backend (for benchmarking / debugging):
VLLM_ATTENTION_BACKEND=FLASHINFER  vllm serve ...
VLLM_ATTENTION_BACKEND=FLASH_ATTN  vllm serve ...
VLLM_ATTENTION_BACKEND=XFORMERS    vllm serve ...

# Python API -- backend visible in engine config
from vllm import LLM
llm = LLM("meta-llama/Meta-Llama-3-8B-Instruct", dtype="float16")
print(llm.llm_engine.model_config.attention_backend)
```

PART B — Paged Attention Internals: Block Manager Deep-Dive

Recap: Block Table Data Structure

The Day 9 topic introduced PagedAttention conceptually. Today we go inside the vLLM source code to understand the exact data structures and eviction policy. This knowledge is essential for debugging KV cache OOM errors and understanding why certain workloads exhaust memory faster than expected.

```
# vLLM block manager data structures (simplified from vllm/core/block_manager.py)
#
# PhysicalTokenBlock:
#   device      : "gpu" | "cpu"
#   block_number: int  (index into the pre-allocated pool)
#   ref_count   : int  (how many sequences share this block -- for CoW)
#   block_size  : int  (tokens per block, e.g. 16)
#
# LogicalTokenBlock:
#   block_number: int  (logical index within a sequence, 0..N)
#   token_ids   : List[int]  (token IDs stored in this block)
#
# BlockTable (per sequence):
#   List[PhysicalTokenBlock]
#   maps logical block index -> physical block
#   e.g. [Block#7, Block#23, Block#2, Block#41]
#        (physical blocks non-contiguous -- that's the whole point)
#
# BlockAllocator:
#   free_blocks  : Deque[PhysicalTokenBlock]  (available GPU blocks)
#   Full blocks  : Set[PhysicalTokenBlock]    (100% filled, can be shared)
#
# Eviction policy: PRIORITISED PREEMPTION
#   When free_blocks is empty and a new sequence needs blocks:
#   1. Swap entire sequences to CPU (move their physical blocks to CPU pool)
#   2. Sequences with LARGEST allocated block count preempted first
#   3. Swapped sequences resumed when GPU blocks become free
#   (This is why --swap-space matters: bigger CPU pool = fewer preemptions)

# Trace the block table for a running sequence (debug mode):
VLLM_LOGGING_LEVEL=DEBUG vllm serve ... 2>&1 | grep -E "block_table|allocat|preempt|swap"
```

Block Allocation Lifecycle

Event	Block Manager Action	Result
Request arrives	Allocate blocks for prompt tokens	N_prompt / block_size blocks reserved
Each decode step	Append 1 token to last block	Block fills up after block_size steps
Block fills up	Allocate 1 new physical block	BlockTable grows by 1 entry
Sequence finishes	Decrement ref_count on all blocks	ref_count=0 -> returned to free pool
Fork (beam search)	Increment ref_count on shared blocks (CoW)	Blocks shared, not copied yet
Diverging fork	Copy-on-write: duplicate block before writing	Shared block cloned for this branch

Memory pressure	Preempt sequence: move GPU->CPU blocks	Sequence swapped, blocks freed
Preempted resumes	Move CPU->GPU blocks, restore BlockTable	Sequence continues from swap

Tracing PagedAttention in vLLM Source Code

```
# Key files to read in the vLLM codebase (github.com/vllm-project/vllm)
#
# vllm/core/block_manager.py         -- BlockAllocator, BlockSpaceManager
# vllm/core/scheduler.py             -- Continuous batching scheduler
# vllm/worker/model_runner.py        -- Executes forward pass with block tables
# vllm/attention/backends/flash_attn.py  -- FlashAttention backend
# vllm/attention/backends/flashinfer.py  -- FlashInfer backend (decode)

# Explore block allocation interactively:
from vllm import LLM, SamplingParams
from vllm.core.block_manager import BlockSpaceManager

llm = LLM("microsoft/phi-2", dtype="float16", gpu_memory_utilization=0.85,
          block_size=16, max_model_len=2048)

# Access the scheduler's block manager
scheduler = llm.llm_engine.scheduler[0]
bm        = scheduler.block_manager

# Inspect GPU block pool state
print(f"GPU blocks total : {bm.get_num_total_gpu_blocks()}")
print(f"GPU blocks free  : {bm.get_num_free_gpu_blocks()}")
print(f"CPU blocks total : {bm.get_num_total_cpu_blocks()}")
print(f"Block size       : {bm.block_size} tokens")
print(f"GPU utilization  : {1 - bm.get_num_free_gpu_blocks()/bm.get_num_total_gpu_blocks():.1%}")
```

Memory Fragmentation Analysis

```
# Internal fragmentation: wasted space in the last (partially filled) block
# External fragmentation: eliminated by PagedAttention (contiguous not needed)
#
# Worst-case internal fragmentation per sequence: block_size - 1 tokens
# With block_size=16 and FP16 KV: 15 tokens wasted = 15 x 2 x num_heads x head_dim x 2 bytes
# For LLaMA-3 8B (32 layers, 8 KV heads, 128 head_dim):
#   = 15 x 2 x 32 x 8 x 128 x 2 bytes = 3.93 MB wasted per sequence
# With 256 concurrent sequences: 256 x 3.93 MB = ~1 GB worst-case fragmentation
#
# This is bounded and predictable -- a major improvement over contiguous allocation
# where fragmentation could prevent new sequences from starting at all.

def analyse_fragmentation(block_size, num_seqs, avg_seq_len,
                          num_layers, num_kv_heads, head_dim, dtype_bytes=2):
    # Internal fragmentation (last block of each sequence)
    avg_waste_tokens  = block_size / 2   # expected value = half a block
    waste_per_seq_mb  = (avg_waste_tokens * 2 * num_layers *
                         num_kv_heads * head_dim * dtype_bytes) / 1e6
    total_waste_mb    = waste_per_seq_mb * num_seqs

    # Total KV cache used (useful)
```

```
    useful_kv_mb      = (avg_seq_len * 2 * num_layers *
                         num_kv_heads * head_dim * dtype_bytes * num_seqs) / 1e6

    frag_pct          = 100 * total_waste_mb / (useful_kv_mb + total_waste_mb)

    print(f"Block size         : {block_size} tokens")
    print(f"Sequences          : {num_seqs}")
    print(f"Avg sequence len   : {avg_seq_len} tokens")
    print(f"Wasted per seq     : {waste_per_seq_mb:.2f} MB")
    print(f"Total wasted       : {total_waste_mb:.1f} MB")
    print(f"Useful KV cache    : {useful_kv_mb:.1f} MB")
    print(f"Fragmentation      : {frag_pct:.1f}%")

# LLaMA-3 8B parameters
analyse_fragmentation(
    block_size=16, num_seqs=64, avg_seq_len=512,
    num_layers=32, num_kv_heads=8, head_dim=128
)
```

PART C — Advanced KV Cache Tuning in vLLM

The Three Tuning Axes

vLLM's KV cache behaviour is controlled by three orthogonal axes that every production engineer must understand and tune for their specific workload:

Axis	Controls	Key Flags
1. Capacity	How much VRAM is dedicated to KV	--gpu-memory-utilization, --max-model-len
2. Reuse (prefix cache)	Whether past KV blocks are reused	--enable-prefix-caching, --enable-chunked-prefill
3. Precision	How many bytes per KV element	--kv-cache-dtype (fp16/fp8)

Axis 1: Capacity -- gpu-memory-utilization

At startup vLLM runs a memory profiling step: it loads the model weights, measures the VRAM used, then allocates the remaining VRAM x gpu_memory_utilization as KV cache blocks. The number of blocks determines the maximum total sequence length that can be served simultaneously.

```
# How vLLM calculates the number of KV cache blocks at startup:
#
# Step 1: Profile peak memory during a dummy forward pass
#   peak_memory = model_weights + activation_buffers + misc
#
# Step 2: Calculate available memory for KV cache
#   gpu_total_memory = torch.cuda.get_device_properties(0).total_memory
#   kv_cache_memory  = gpu_total_memory * gpu_memory_utilization - peak_memory
#
# Step 3: Calculate block count
#   kv_cache_block_size_bytes = block_size * num_layers * 2 * num_kv_heads
#                               * head_dim * dtype_bytes
#   num_gpu_blocks = kv_cache_memory // kv_cache_block_size_bytes
#
# For LLaMA-3 8B on A100 80GB (gpu_memory_utilization=0.90):
#   model_weights    ~ 16 GB (FP16)
#   activation buffer~ 2 GB (estimated)
#   kv_cache_memory  = 80 * 0.90 - 18 = 54 GB
#   block_size_bytes = 16 * 32 * 2 * 8 * 128 * 2 = 2,097,152 bytes = 2 MB/block
#   num_gpu_blocks   = 54 GB / 2 MB = 27,648 blocks
#   max_total_tokens = 27,648 * 16 = 442,368 tokens concurrently in cache

def estimate_kv_blocks(gpu_vram_gb, model_size_gb, gpu_memory_utilization,
                       block_size, num_layers, num_kv_heads, head_dim,
                       dtype_bytes=2, activation_buffer_gb=2):
    kv_mem_gb   = gpu_vram_gb * gpu_memory_utilization - model_size_gb - activation_buffer_gb
    block_bytes = block_size * num_layers * 2 * num_kv_heads * head_dim * dtype_bytes
    num_blocks  = int(kv_mem_gb * 1e9 // block_bytes)
    max_tokens  = num_blocks * block_size
    print(f"KV cache memory   : {kv_mem_gb:.1f} GB")
    print(f"Bytes per block   : {block_bytes / 1e6:.2f} MB")
    print(f"GPU blocks        : {num_blocks:,}")
    print(f"Max cached tokens : {max_tokens:,}")
```

```
    return num_blocks

# LLaMA-3 8B on A100 80GB
print("=== LLaMA-3 8B, A100 80GB, util=0.90 ===")
estimate_kv_blocks(80, 16, 0.90, 16, 32, 8, 128)

print("
=== LLaMA-3 8B, A100 80GB, util=0.90, FP8 KV ===")
estimate_kv_blocks(80, 16, 0.90, 16, 32, 8, 128, dtype_bytes=1)   # 2x more blocks!
```

Axis 2: Prefix Caching (Automatic Prefix Cache)

Prefix caching (APC) allows vLLM to reuse KV blocks computed for a shared prompt prefix across multiple requests. When enabled, vLLM hashes each filled block's token IDs. On a cache hit, the block's KV data is reused directly -- the prefill computation is skipped for that block entirely. This is the single highest-impact optimisation for workloads with shared system prompts, RAG documents, or tool schemas -- which covers the vast majority of production agent deployments.

```
# Enable prefix caching in vLLM server
vllm serve meta-llama/Meta-Llama-3-8B-Instruct \
    --enable-prefix-caching \          # enable APC
    --dtype float16 \
    --max-model-len 8192

# Chunked prefill (required for APC to work efficiently with long prompts)
vllm serve meta-llama/Meta-Llama-3-8B-Instruct \
    --enable-prefix-caching \
    --enable-chunked-prefill \         # process long prompts in chunks
    --max-num-batched-tokens 2048 \    # tokens per chunked prefill step
    --dtype float16

# Python API
from vllm import LLM, SamplingParams

llm = LLM(
    model="meta-llama/Meta-Llama-3-8B-Instruct",
    enable_prefix_caching=True,
    enable_chunked_prefill=True,
    max_num_batched_tokens=2048,
    dtype="float16",
)
```

Measuring Prefix Cache Hit Rate

```
import requests, time, json
from openai import OpenAI

client = OpenAI(base_url="http://localhost:8000/v1", api_key="vllm")

# Craft a long shared system prompt (simulates RAG document or tool schema)
SYSTEM_PROMPT = (
    "You are a senior AI infrastructure engineer. "
    "Tools: vllm_bench, prometheus_query, kubectl_exec, load_test. "
    "Reply with tool name + params in JSON before prose. "
) * 30   # inflate to ~500 tokens to make cache benefit visible
```

```
questions = [
    "How do I measure KV cache utilisation?",
    "What Prometheus metric shows GPU memory usage?",
    "How do I check how many requests are currently running?",
    "What is the command to see vLLM block allocation?",
    "How do I benchmark token throughput?",
]

ttfts = []
for i, q in enumerate(questions):
    t0 = time.perf_counter()
    resp = client.chat.completions.create(
        model="meta-llama/Meta-Llama-3-8B-Instruct",
        messages=[
            {"role": "system", "content": SYSTEM_PROMPT},
            {"role": "user",   "content": q},
        ],
        max_tokens=80,
        stream=True,
    )
    ttft = None
    for chunk in resp:
        if chunk.choices[0].delta.content and ttft is None:
            ttft = time.perf_counter() - t0
    ttfts.append(ttft)
    label = "cold" if i == 0 else f"warm ({i})"
    print(f"  {label:<12}: TTFT = {ttft:.3f}s")

print(f"
Cold TTFT   : {ttfts[0]:.3f}s  (prefix not cached)")
print(f"Warm avg    : {sum(ttfts[1:])/len(ttfts[1:]):.3f}s  (prefix cached)")
print(f"TTFT speedup: {ttfts[0] / (sum(ttfts[1:])/len(ttfts[1:])):.1f}x")

# Read cache metrics directly from Prometheus
metrics = requests.get("http://localhost:8000/metrics").text
for line in metrics.split("
"):
    if "cache_hit" in line.lower() and not line.startswith("#"):
        print(f"  {line.strip()}")
```

Axis 3: FP8 KV Cache -- Halving KV Memory

The KV cache is stored in FP16 by default (2 bytes per element). vLLM 0.3+ supports FP8 KV cache quantisation -- storing K and V tensors in 8-bit float format (1 byte per element). This doubles the number of KV blocks available without increasing VRAM, effectively doubling the maximum concurrent sequence length or batch size. Quality impact is minimal: the KV values are quantized per-token with per-channel scaling factors stored alongside.

```
# Enable FP8 KV cache (requires Ampere+ GPU, vLLM >= 0.3)
vllm serve meta-llama/Meta-Llama-3-8B-Instruct \
    --kv-cache-dtype fp8 \             # or fp8_e4m3 / fp8_e5m2
    --dtype float16 \                  # model weights still in FP16
    --gpu-memory-utilization 0.90

# Python API
```

```
from vllm import LLM

llm_fp16_kv = LLM("meta-llama/Meta-Llama-3-8B-Instruct",
                  dtype="float16", kv_cache_dtype="auto")   # FP16 KV

llm_fp8_kv  = LLM("meta-llama/Meta-Llama-3-8B-Instruct",
                  dtype="float16", kv_cache_dtype="fp8")    # FP8 KV

# Compare KV block counts (FP8 should have ~2x more blocks)
for label, engine in [("FP16 KV", llm_fp16_kv), ("FP8 KV", llm_fp8_kv)]:
    scheduler = engine.llm_engine.scheduler[0]
    n_blocks  = scheduler.block_manager.get_num_total_gpu_blocks()
    print(f"{label}: {n_blocks:,} GPU blocks = {n_blocks * 16:,} max cached tokens")
```

Complete Tuning Decision Tree

```
# When to apply each optimisation:
#
# 1. Flash Attention (always on in vLLM -- verify with VLLM_ATTENTION_BACKEND)
#    Benefit : 2-4x prefill speedup, lower VRAM peak during attention
#    Cost    : None on Ampere+ GPU
#    Action  : Confirm "FlashInfer" or "Flash Attention" in vLLM startup logs
#
# 2. --gpu-memory-utilization (default 0.90)
#    Too low  -> few KV blocks -> low concurrency -> requests queue up
#    Too high -> OOM during spikes -> server crashes
#    Sweet spot: 0.85-0.92 for most workloads
#    Action  : Start at 0.90, lower if you see CUDA OOM, raise if cache usage <60%
#
# 3. --enable-prefix-caching
#    Use when: shared system prompts, RAG documents, tool schemas, agent loops
#    Skip when: every request is completely unique (no shared prefix)
#    Measure : TTFT reduction on warm requests (expect 2-10x for long shared prefix)
#
# 4. --kv-cache-dtype fp8
#    Use when: VRAM-constrained, concurrency limited by KV cache size
#    Skip when: quality-critical tasks (measure PPL degradation first)
#    Benefit : ~2x more KV blocks -> ~2x higher max concurrent sequences
#
# 5. --max-model-len (reduce from default)
#    Use when: you know requests won't exceed a shorter context (e.g. 4096 vs 8192)
#    Benefit : More VRAM stays as KV cache (model doesn't pre-allocate position buffers)
#    Risk    : Requests exceeding max_model_len are rejected with 400 error

# Recommended starting config for production 8B model on single A100 80GB:
PRODUCTION_CONFIG = {
    "model":                    "meta-llama/Meta-Llama-3-8B-Instruct",
    "dtype":                    "float16",
    "gpu_memory_utilization":   0.90,
    "max_model_len":            8192,
    "enable_prefix_caching":    True,
    "enable_chunked_prefill":   True,
    "max_num_batched_tokens":   4096,
    "kv_cache_dtype":           "auto",    # fp8 if concurrency limited
    "block_size":               16,
    "swap_space":               4,         # GB CPU swap
```

```
    "max_num_seqs":             256,
}
```

Testing & Metrics

Comprehensive Three-Optimization Benchmark

```
#!/usr/bin/env python3
# benchmark_attention_opts.py
# Measures the combined impact of FA2 + PagedKV + prefix caching

import time, asyncio, httpx, requests
from transformers import AutoModelForCausalLM, AutoTokenizer
import torch

MODEL_ID  = "microsoft/phi-2"
N_PROMPTS = 20
N_TOKENS  = 100
SHARED_PREFIX = "You are an expert AI engineer. " * 30  # ~180 token shared prefix

# ■■ Stage 1: HF naive vs FA2 (prefill speed) ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
print("Stage 1: Flash Attention speedup on prefill...")
tok = AutoTokenizer.from_pretrained(MODEL_ID)

for impl, label in [("eager","Standard"), ("flash_attention_2","FlashAttn2")]:
    try:
        m = AutoModelForCausalLM.from_pretrained(
            MODEL_ID, torch_dtype=torch.float16, device_map="cuda",
            attn_implementation=impl)
        m.eval()
        long_prompt = SHARED_PREFIX + "Explain KV caching in detail."
        inp = tok(long_prompt, return_tensors="pt").to("cuda")
        torch.cuda.synchronize()
        t0  = time.perf_counter()
        with torch.no_grad():
            m(**inp)   # prefill only -- no generation
        torch.cuda.synchronize()
        prefill_ms = (time.perf_counter() - t0) * 1000
        print(f"  {label:<20}: prefill {prefill_ms:.1f}ms  "
              f"({inp['input_ids'].shape[1]} tokens)")
        del m; torch.cuda.empty_cache()
    except Exception as e:
        print(f"  {label}: skipped ({e})")

# ■■ Stage 2: vLLM prefix cache hit rate ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
print("
Stage 2: Prefix cache effect on TTFT...")
print("(Assumes vLLM running at localhost:8000 with --enable-prefix-caching)")

questions = ["What is TTFT?", "Explain TBT.", "What is throughput?",
             "Define KV cache.", "What is block eviction?"]
results = []
for i, q in enumerate(questions):
    t0   = time.perf_counter()
    resp = requests.post("http://localhost:8000/v1/chat/completions", json={
        "model": "meta-llama/Meta-Llama-3-8B-Instruct",
        "messages": [{"role":"system","content": SHARED_PREFIX},
                     {"role":"user",  "content": q}],
```

```
        "max_tokens": 50, "stream": False,
    })
    elapsed = time.perf_counter() - t0
    results.append(("cold" if i==0 else "warm", elapsed))
    print(f"  {'cold' if i==0 else 'warm'}: {elapsed:.3f}s")

cold_ttft = results[0][1]
warm_avg  = sum(r[1] for r in results[1:]) / (len(results)-1)
print(f"  Cache speedup: {cold_ttft/warm_avg:.1f}x")

# ■■ Stage 3: FP8 vs FP16 KV blocks ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
print("
Stage 3: KV cache block counts (FP16 vs FP8)...")
print("(Restart vLLM with --kv-cache-dtype fp8 and re-check /metrics)")
metrics = requests.get("http://localhost:8000/metrics").text
for line in metrics.split("
"):
    if "cache_usage" in line or "num_blocks" in line:
        if not line.startswith("#") and line.strip():
            print(f"  {line.strip()}")
```

Prometheus Metrics Reference for Attention Tuning

Metric	What it measures	Healthy range
vllm:gpu_cache_usage_perc	Fraction of KV blocks in use	0.60 - 0.85
vllm:cpu_cache_usage_perc	CPU swap pool utilisation	< 0.20 (swapping = bad)
vllm:num_requests_running	Sequences in current batch	Stable, not oscillating
vllm:num_requests_waiting	Requests queued for KV blocks	Should trend to 0 at low load
vllm:num_requests_swapped	Sequences on CPU (preempted)	0 ideally; spikes ok
vllm:time_to_first_token_seconds	TTFT histogram (p50/p95/p99)	<1s p99 for 8B GPU
vllm:time_per_output_token_seconds	TBT histogram (decode speed)	<30ms p50 for 8B
vllm:prefix_cache_hit_rate	Fraction of prefill tokens from cache	>0.5 if prefix caching on
vllm:spec_decode_acceptance_rate	Draft token acceptance (spec decoding)	>0.7 for net speedup

Shell Verification Commands

```
# 1. Confirm Flash Attention backend
vllm serve ... 2>&1 | grep -iE "attention backend|flashinfer|flash_attn"

# 2. Check KV cache block counts at startup
vllm serve ... 2>&1 | grep -E "GPU blocks|CPU blocks|block size"
# Expected: "# GPU blocks: 27648, # CPU blocks: 2048"

# 3. Live Prometheus monitoring (single command dashboard)
watch -n 2 'curl -s http://localhost:8000/metrics | grep -E "cache_usage|num_requests|ttft|tpot" | grep -v "^#"'

# 4. Prefix cache effectiveness (before and after enabling)
# Before: Run 5 requests with same prefix, measure average TTFT
```

```
for i in $(seq 5); do
    curl -s -w "%{time_total}
" -o /dev/null \
        http://localhost:8000/v1/chat/completions \
        -H "Content-Type: application/json" \
        -d '{"model":"llama3","messages":[{"role":"system","content":"LONG_SHARED_PREFIX"},{"role":"user","conte
done

# 5. Estimate HBM bandwidth savings from Flash Attention (via nsight)
ncu --metrics l1tex__t_bytes_pipe_lsu_mem_global_op_ld.sum,\
             l1tex__t_bytes_pipe_lsu_mem_global_op_st.sum \
    python my_attention_script.py
```

Key Takeaways

• **Flash Attention eliminates the N^2 HBM bottleneck** by tiling Q/K/V to fit entirely in SRAM and never materialising the full attention score matrix. HBM traffic drops from O(N^2) to O(Nd) -- a 17x reduction for 8K context, translating to 2-4x real prefill speedup.
• **Flash Attention is automatic in vLLM** (FlashInfer for decode, FA2 for prefill). Confirm the backend at startup and never override it without a benchmarked reason.
• **The PagedAttention block manager** maintains a block table per sequence, enabling non-contiguous KV allocation, copy-on-write sharing for beam search, and priority-based preemption under memory pressure. Internal fragmentation is bounded to at most block_size-1 tokens per sequence.
• **--enable-prefix-caching** is the highest-ROI single flag for agent and RAG workloads. Warm TTFT improvements of 2-10x are typical for 500+ token shared prefixes.
• **FP8 KV cache doubles available KV blocks** (2 bytes -> 1 byte per KV element) with minimal quality impact -- use it whenever concurrency is limited by KV cache capacity rather than model compute.
• **Flash Attention + PagedKV + prefix caching together** are the three mechanisms behind vLLM's 5-10x throughput advantage over naive HuggingFace inference. Each is independently valuable; combined, they compound.

Next topic: **Day 11 -- Distributed & Parallel Inference**: tensor parallelism with vLLM (--tensor-parallel-size), pipeline parallelism across nodes, GPU-aware Kubernetes deployment, and multi-node VRAM budget calculation.